

Intel·ligència Artificial

PRÀCTICA I

Mario VILAR

20392724

David Díez

20392643

2 d'octubre de 2023



UNIVERSITAT DE
BARCELONA

ÍNDEX

1	Introducció	2
2	Cos del treball	2
3	Conclusions	6

INTRODUCCIÓ

L'objectiu d'aquesta pràctica és familiaritzar-se amb els algorismes de cerca bàsics de la intel·ligència artificial, en particular, de l' A^* . Amb aquesta finalitat, la nostra tasca consisteix a implementar un algoritme de cerca d'IA per aconseguir objectius específics en un tauler d'escacs.

En aquesta pràctica, es busca que adquirim coneixements sobre els fonaments dels escacs i la manera com els algorismes de cerca d'IA es poden aplicar a aquest joc. Mitjançant la implementació d'un algoritme de cerca, s'exploraran conceptes com la generació de moviments, l'avaluació de posicions i la presa de decisions estratègiques en el context de l'escacs. L'enfocament estarà en el desenvolupament de habilitats pràctiques que permetin a la IA prendre decisions efectives en el joc d'escacs, la qual cosa és fonamental per al desenvolupament de sistemes de joc d'escacs automatitzats i altres tipus d'aplicacions d'IA relacionades amb jocs i problemes de presa de decisions.

El simulador consta d'una jerarquia de quatre classes: `aichess`, `chess`, `board`, `piece`. Els tres últims implementen la dinàmica d'una partida d'escacs, que pot ser dirigida per dos jugadors. Si us plau, executeu el principal a la classe d'escacs amb aquesta finalitat. Finalment, la classe `Aichess` és una càpsula de les tres anteriors, amb la finalitat d'implementar algorismes d'IA per analitzar i alterar la dinàmica del joc d'escacs.

COS DEL TREBALL

Exercici 2.1. *You are instructed to find that state by implementing a basic algorithm.*

1. *Please do provide a list of the visited states from the origin to the target (3p) and the minimal depth necessary to reach the target (1p).*
2. *A^* Search (3p).*

Explicació. Hem creat un codi que llista tots els estats visitats, i la mínima profunditat necessària per arribar al nostre objectiu. Notem que aquesta mínima profunditat no tindria per què ser la mateixa que la del camí amb mínim cost, en el cas que estiguéssim en un arbre amb *distàncies* entre nodes amb *valors* diferents entre ells. *Per exemple, en l'exemple de les rutes a Teoria.* En el cas d'un taulell d'escacs, però, la g (cost des del node d'origen fins el node n) augmenta d'un en un i, doncs, podem assegurar que la mínima profunditat serà la mateixa que la del camí amb mínim cost.

El conjunt de nodes visitats que el nostre programa ha visitat ha sigut el següent:

```

1  [[7, 0, 2], [7, 4, 6]], [[6, 0, 2], [7, 4, 6]], [[5, 0, 2], [7, 4, 6]], [[4, 0, 2], [7, 4,
2  ↪ 6]], [[3, 0, 2], [7, 4, 6]], [[2, 0, 2], [7, 4, 6]], [[1, 0, 2], [7, 4, 6]], [[0, 0,
3  ↪ 2], [7, 4, 6]], [[7, 1, 2], [7, 4, 6]], [[7, 2, 2], [7, 4, 6]], [[7, 3, 2], [7, 4, 6]],
4  ↪ [[7, 3, 6], [7, 0, 2]], [[6, 3, 6], [7, 0, 2]], [[6, 4, 6], [7, 0, 2]], [[6, 5, 6], [7,
5  ↪ 0, 2]], [[7, 5, 6], [7, 0, 2]], [[0, 1, 2], [7, 4, 6]], [[0, 2, 2], [7, 4, 6]], [[0, 3,
6  ↪ 2], [7, 4, 6]], [[0, 4, 2], [7, 4, 6]], [[7, 3, 6], [0, 0, 2]], [[6, 3, 6], [0, 0, 2]],
7  ↪ [[6, 4, 6], [0, 0, 2]], [[6, 5, 6], [0, 0, 2]], [[7, 5, 6], [0, 0, 2]], [[7, 2, 6], [0,
8  ↪ 0, 2]], [[6, 2, 6], [0, 0, 2]], [[5, 2, 6], [0, 0, 2]], [[5, 3, 6], [0, 0, 2]], [[5, 4,
9  ↪ 6], [0, 0, 2]], [[1, 0, 2], [6, 3, 6]], [[2, 0, 2], [6, 3, 6]], [[3, 0, 2], [6, 3, 6]],
10 ↪ [[4, 0, 2], [6, 3, 6]], [[5, 0, 2], [6, 3, 6]], [[6, 0, 2], [6, 3, 6]], [[0, 1, 2], [6,
11 ↪ 3, 6]], [[0, 2, 2], [6, 3, 6]], [[0, 3, 2], [6, 3, 6]], [[0, 4, 2], [6, 3, 6]], [[6, 1,
12 ↪ 6], [0, 0, 2]], [[5, 1, 6], [0, 0, 2]], [[4, 1, 6], [0, 0, 2]], [[4, 2, 6], [0, 0, 2]],
13 ↪ [[4, 3, 6], [0, 0, 2]], [[1, 0, 2], [5, 2, 6]], [[2, 0, 2], [5, 2, 6]], [[3, 0, 2], [5,
14 ↪ 2, 6]], [[4, 0, 2], [5, 2, 6]], [[5, 0, 2], [5, 2, 6]], [[6, 0, 2], [5, 2, 6]], [[7, 0,
15 ↪ 2], [5, 2, 6]], [[0, 1, 2], [5, 2, 6]], [[0, 2, 2], [5, 2, 6]], [[0, 3, 2], [5, 2, 6]],
16 ↪ [[0, 4, 2], [5, 2, 6]], [[3, 1, 6], [0, 0, 2]], [[3, 2, 6], [0, 0, 2]], [[3, 3, 6], [0,
17 ↪ 0, 2]], [[1, 0, 2], [4, 2, 6]], [[2, 0, 2], [4, 2, 6]], [[3, 0, 2], [4, 2, 6]], [[4, 0,
18 ↪ 2], [4, 2, 6]], [[5, 0, 2], [4, 2, 6]], [[6, 0, 2], [4, 2, 6]], [[7, 0, 2], [4, 2, 6]],
19 ↪ [[0, 1, 2], [4, 2, 6]], [[0, 2, 2], [4, 2, 6]], [[0, 3, 2], [4, 2, 6]], [[0, 4, 2], [4,
20 ↪ 2, 6]], [[2, 2, 6], [0, 0, 2]], [[2, 3, 6], [0, 0, 2]], [[2, 4, 6], [0, 0, 2]], [[3, 4,
21 ↪ 6], [0, 0, 2]], [[4, 4, 6], [0, 0, 2]], [[1, 0, 2], [3, 3, 6]], [[2, 0, 2], [3, 3, 6]],
22 ↪ [[3, 0, 2], [3, 3, 6]], [[4, 0, 2], [3, 3, 6]], [[5, 0, 2], [3, 3, 6]], [[6, 0, 2], [3,
23 ↪ 3, 6]], [[7, 0, 2], [3, 3, 6]], [[0, 1, 2], [3, 3, 6]], [[0, 2, 2], [3, 3, 6]], [[0, 3,
24 ↪ 2], [3, 3, 6]], [[0, 4, 2], [3, 3, 6]]

```

A més a més, el valor de depth ha sigut de 6. Per alta banda, el codi que per implementar l'algorisme A^* és el següent:

```

1  def AStarSearch(self, currentState):
2      # List of the nodes we haven't seen yet. The order depends on h() function,
3      # then we must use a min heap
4      frontera = []
5      heapify(frontera)
6
7      # This is the dictionary regarding the path to the target state.
8      # We create a dict entry for the currentState, in this case the root node.
9      # Indeed, the parent state must be None and the current depth, -1.
10     self.dictPath[str(currentState)] = (None, -1)
11
12     # Set the currentState depth after the root.
13     depthCurrentState = 0
14
15     # Calculate the heuristic of our currentState
16     h = self.h(currentState)
17

```

```

18  # We have to create a node structure, and we choose to do so with a 4-tuple.
19  # [0] - f function. Important to establish an order between frontera items.
20  # [1] - currentState. Doesn't need much of an explanation.
21  # [2] - g function. Accumulated cost before currentState.
22  # [3] - h function. The heuristic of currentState.
23  initial_node = (0+h,currentState,0,h)
24
25  # Using frontera as a min-heap, we push the initial node and append the first visited
    ↪ state.
26  heappush(frontera,initial_node)
27  self.listVisitedStates.append(currentState)
28
29  # We iterate while our frontera is not empty.
30  while frontera:
31
32      # Retrieve the best state in the heap. We don't care about the other values (yet).
33      f, state, g, h = heappop(frontera)
34
35      # Get the depth of the best item's state.
36      depthState = self.dictPath[str(state)][1] + 1
37
38      # Beforehand we fixed a max depth, probably in order to avoid maximum depth errors.
39      if depthState > self.depthMax:
40          break
41
42      # We move the pieces every time we can.
43      if depthState > 0:
44          # This function needs: currentState, depthCurrentState, state, depthState
45          self.movePieces(currentState, depthCurrentState, state, depthState)
46
47      # Check against the isCheckMate function if we arrived at desired output.
48      if self.isCheckMate(state):
49          # Reconstruct the path with the help of auxiliar function.
50          # Parent nodes are essential. We set their values later on.
51          self.reconstructPath(state, depthState)
52          # We want to keep all the visited states and print the current depth
53          print("VISITED STATES: " + str(self.listVisitedStates))
54          print("MINIMAL DEPTH: " + str(depthState))
55          break
56
57      # On the other hand we iterate through the next states.
58      for sucesorState in self.getListNextStatesW(state):
59          # We update the h, g and f values for the new state
60          new_h = self.h(sucesorState)
61          new_g = g + 1
62          new_f = new_h + new_g
63

```

```

64     # Important condition. Mildly tweaked, but in essence the same as the one
65     # in slides pseudocode.
66     if (not self.isVisited(sucesorState)) or (self.isVisited(sucesorState)\
67     and new_f < f):
68         # Mark state as visited, put it in frontera and set the target path
69         # ↪ accordingly.
70         self.listVisitedStates.append(sucesorState)
71         heappush(frontera, (new_f, sucesorState, new_g, new_h))
72         self.dictPath[str(sucesorState)] = (state, depthState)
73
74     # Prepare for another iteration
75     currentState = state
76     depthCurrentState = depthState
77
78     return 0

```

Aquest codi implementa l'algorisme A^* per resoldre un problema de cerca en un espai d'estats. Es comença amb un estat inicial i es busca trobar un estat objectiu. L'algorisme manté un *minheap* (*frontera*) per gestionar els estats pendents de revisar.

Es calculen les funcions f , g i h per a cada estat i es selecciona l'estat amb la puntuació f més baixa per explorar següents. Es verifica si s'ha arribat a l'estat objectiu i, en cas afirmatiu, es reconstrueix el camí. En cas contrari, s'exploraran els estats successors possibles, es calculen les seves puntuacions i es decideix si s'afegeixen a la frontera. Aquest procés es repeteix fins a trobar l'estat objectiu o es supera una profunditat màxima predefinida.

Fent referència al correu enviat per l'Adrian Fernandez Cid el dia 2 d'Octubre del 2023 a les 09:11 volíem comentar que nosaltres sí que teníem la condició dels costos implementada. La nostra motivació era donar-li robustesa al codi i un millor funcionament. Val a dir que ens va donar una sèrie de problemes però, al final, vam resoldre amb èxit. A més, nosaltres tampoc extraïem el node *car* de la frontera ja que com bé es diu no suposa un problema: en general aquell node no sortirà de la frontera. ■

Exercici 2.2. *Now, initialize the board with the white king on position [7,7] and rerun your code. Does your algorithm work in precisely the same way? Did you have to change anything? (3p)*

Explicació. La pregunta no és del tot clara: un algorisme es basa justament en la rigidesa de la seva estructura, de manera que treballa d'una sola forma per definició. Pel que entenem, l'algorisme no treballa exactament de la mateixa manera en el sentit que la seqüència de moviments no és la mateixa, ja que hem canviat el rei blanc de posició. Sorprèn, però:

- Que el primer moviment és sempre el de la torre cap a la cantonada esquerra superior.
- Que cap de les dues rutes és la *més directa*. En quant a cost per f , evidentment, són equivalents perquè estem treballant amb una distància de Manhattan, però el rei agafa moltes diagonals innecessàries en

el primer cas i en el segon *passa la columna de llarg* amb el moviment $[3, 3]$ per després tornar a la corresponent, $[2, 4]$. La raó d'aquest fet no es res més que quan el rei arriba a la posició $[4, 4]$ pot moure's cap a $[3, 3]$, $[3, 4]$ o $[3, 5]$ i tots tres estats tenen la mateixa heurística, per tant, el rei escull de forma aleatòria una d'aquestes posicions.

- En aquest sentit, sembla que tots dos coincideixen en aquest estat $[[3, 3, 6], [0, 0, 2]]$.

Val la pena esmentar que és irrellevant el camí que prengui sempre i quant el cost sigui el mateix i, de fet, la profunditat que assolirà en l'arbre de cerca serà exactament la mateixa en qualsevol dels camins de mateix cost.

```
1  [[7, 0, 2], [7, 4, 6]], [[0, 0, 2], [7,
   ↪ 4, 6]], [[6, 3, 6], [0, 0, 2]], [[5,
   ↪ 2, 6], [0, 0, 2]], [[4, 2, 6], [0,
   ↪ 0, 2]], [[3, 3, 6], [0, 0, 2]], [[2,
   ↪ 4, 6], [0, 0, 2]]]
```

Figura 1: Seqüència de moviments guanyadora feta per a $K=[7, 4]$.

```
1  [[7, 0, 2], [7, 7, 6]], [[0, 0, 2], [7,
   ↪ 7, 6]], [[6, 6, 6], [0, 0, 2]], [[5,
   ↪ 5, 6], [0, 0, 2]], [[4, 4, 6], [0,
   ↪ 0, 2]], [[3, 3, 6], [0, 0, 2]], [[2,
   ↪ 4, 6], [0, 0, 2]]]
```

Figura 2: Seqüència de moviments guanyadora feta per a $K=[7, 7]$.

Responent, doncs, a la pregunta: no sembla que canviant l'estat millorem el rendiment del nostre codi.

Per altra banda, és interessant comentar que en cas d'haver mogut el rei negre sí hauríem d'haver canviat el codi, en particular, la funció que calcula l'heurística. El motiu és clar, s'utilitza que el rei blanc ha d'acabar a la posició $(2, 4)$ ja que és on pot fer *escac i mat* al rei negre. D'aquesta manera, si canviem de posició el rei negre aquesta posició deixarà de ser vàlida. ■

CONCLUSIONS

En aquesta pràctica hem fet els nostres primers passos en el món de la Intel·ligència Artificial, hem après una mica més sobre els escacs i aprofundit, en particular, en com funciona l'algorisme A^* .

El que més ens ha costat de la pràctica ha sigut habitar-nos a treballar amb les classes que ja ens venien donades, ja que les quatre tenien moltes funcions i hem hagut de discernir quines eren importants i amb quines no treballaríem *directament*. Considerem que ha sigut un encert que aquest any s'incloguessin exemples d'algorismes ja coneguts com DFS, ja que ens ha permès fet tot aquest procés que acabem de comentar molt més ràpidament.

Al principi vam intentar fer una versió molt més *collita pròpia* de l'algorisme, amb la nostra pròpia llista `alcanzados` i una funció pròpia per comprovar si aquell estat ja estava dins la nostra `alcanzados`, però ens vam adonar que el propi programa l'incorporava amb `self.listVisitedStates`. Ens vam adonar, també, que havíem d'efectuar el moviment amb `movePieces` per generar altres estats i que, per tant, necessitàvem

dotar el nostre codi d'un paràmetre `depth`. Podríem dir, en resum, que el més complex ha sigut interpretar el pseudocodi de teoria per aplicar-lo correctament sobre el codi base de la pràctica, i entendre aquest últim correctament per adaptar-lo a les nostres necessitats.